

AI Applications Lecture 10

Fundamentals of Modern AI Tuning

SUZUKI, Atsushi^a

^aI thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations Revisited
3. Key in efficient parameter selection on a large NN? - Gradient descent.
4. AdamW, Muon, ... Why do we need to learn so many?
5. Can we compute a gradient on a complicated NN? - General backpropagation
6. Summary

Machine Learning Framework: Learning and Inference

The machine learning framework used in many generative AIs is divided into two steps:

- the **learning** step, which appropriately determines the parameters of a parametric function, and
- the **inference** step, which actually executes the desired task using the function determined by the parameters set in the learning step.

This course has so far focused on the inference step, but starting from this lecture, we will focus on “learning”.

Throughout the broad field of generative AI, when we want to consider the problem of how to learn, that is, how to determine the parameters, one non-trivial difficulty is that, as we have seen, the graph structure of the neural network, i.e., the function system, is completely different depending on the type of data being

Different Objective Functions in Different Components

Furthermore, the objective function also differs depending on the field and component, even for the same type of data.

Examples of objective functions:

- **Text field:** Cross entropy when a neural network outputs a probability distribution over a set of tokens (see [1]).
- **Image generation:**
 - Contrastive learning loss by CLIP,
 - Squared error in the noise estimator,
 - Regularized reconstruction squared error in the VAE (in reality, it is more complex than this) [3, 5].

Even though the function systems and objective functions differ this much depending on the field and component, as long as feedforward neural networks are used, **all parameter determination can be done by backpropagation and stochastic gradient methods**. This is the wonderful flexibility of neural networks

Learning Outcomes

At the end of this lecture, students will be able to:

- Find the optimal parameter vector of a modern NN defined on a general directed acyclic multigraph using **backpropagation** and a **stochastic gradient descent** variant.
- Explain why it is impossible to select an optimization algorithm only from theories in modern AI application situations.
- Explain the idea of practically used optimization algorithms, e.g., AdamW and Muon.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations Revisited

3. Key in efficient parameter selection on a large NN? - Gradient descent.

4. AdamW, Muon, ... Why do we need to learn so many?

5. Can we compute a gradient on a complicated NN? - General backpropagation

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations Revisited

3. Key in efficient parameter selection on a large NN? - Gradient descent.

4. AdamW, Muon, ... Why do we need to learn so many?

5. Can we compute a gradient on a complicated NN? - General backpropagation

6. Summary

Mathematical Notations (1): Definitions and Sets

We briefly revisit the basic mathematical notations.

- **Definition:**

- $(\text{LHS}) := (\text{RHS})$: Indicates that the left-hand side is defined by the right-hand side. For example, $a := b$ indicates that a is defined by b .

- **Set:**

- Sets are often denoted by uppercase calligraphic letters. Example: $\mathcal{A}$.
- $x \in \mathcal{A}$: x belongs to $\mathcal{A}$.
- $\{\}$: The empty set.
- $\{a, b, c\}$: The set consisting of elements a, b, c (roster notation).
- $\{x \in \mathcal{A} \mid P(x)\}$: The set of elements in $\mathcal{A}$ for which the proposition $P(x)$ is true (set-builder notation).
- $\mathbb{R}$: The set of all real numbers.
- $\mathbb{R}_{>0}$: The set of all positive real numbers.
- $\mathbb{R}_{\geq 0}$: The set of all non-negative real numbers.
- $\mathbb{Z}$: The set of all integers.
- $\mathbb{N}$: The set of all positive integers.

Mathematical Notations (2): Functions, Vectors, Matrices, Tensors

- **Function:**

- $f : \mathcal{X} \rightarrow \mathcal{Y}$: f maps elements of $\mathcal{X}$ to $\mathcal{Y}$.
- $y = f(x)$: The output is $y \in \mathcal{Y}$ when $x \in \mathcal{X}$ is input to f .

- **Vector:**

- A vector is a column of numbers arranged vertically.
- Vectors are denoted by bold italic lowercase letters: $\boldsymbol{v}$.
- $\boldsymbol{v} \in \mathbb{R}^n$: $\boldsymbol{v}$ is an n -dimensional real vector.
- The i -th element is v_i :

$$\boldsymbol{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}.$$

- **Matrix:**

- Matrices are bold italic uppercase letters: $\boldsymbol{A}$.
- $\boldsymbol{A} \in \mathbb{R}^{m,n}$: an $m \times n$ real matrix.

- The (i, j) -th element is a_{ij} .

Outline

1. Introduction

2. Preliminaries: Mathematical Notations Revisited

3. Key in efficient parameter selection on a large NN? - Gradient descent.

4. AdamW, Muon, ... Why do we need to learn so many?

5. Can we compute a gradient on a complicated NN? - General backpropagation

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations Revisited

3. Key in efficient parameter selection on a large NN? - Gradient descent.

4. AdamW, Muon, ... Why do we need to learn so many?

5. Can we compute a gradient on a complicated NN? - General backpropagation

6. Summary

Question: How do we

Question: How do we select a good parameter vector θ^* ?

Parametric Functions and Objective Functions

AI aims to obtain some appropriate input-output relationship using a **parametric function**. A parametric function $f_{(.)}$ is a mathematical object such that giving a parameter θ determines one function f_{θ} .

Ultimately, what we determine is the parameter vector θ .

To determine θ , we must provide information to the computer (directly or indirectly) about what kind of parameter is desirable for solving the task.

Objective function:

- Takes the parameter as an argument.
- Returns a real value representing its “goodness”.

Maximization, or minimization?

Remark

In many cases in AI, the objective function is designed so that the **smaller** its value, the better the parameter, so it can also be called a **cost function**.

Even if a larger value means better parameters, we can multiply by -1 and treat it as a cost function. Thus, we lose no generality by considering only the case where “smaller is better”. In this lecture, we will follow this convention.

Example of an Objective Function

Example

Suppose we want to create a prediction function from an input space $\mathcal{X} \subset \mathbb{R}^{d_{\text{in}}}$ to an output space $\mathcal{Y} \subset \mathbb{R}^{d_{\text{out}}}$ using a parametric function $f_{(\cdot)}$.

If we have past data $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})_{i=1}^m$, we can embed the desire for good predictions into a squared error function, and design the objective function as

$$\mathcal{L}(\boldsymbol{\theta}) := \frac{1}{m} \sum_{i=1}^m \left\| \mathbf{y}^{(i)} - f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right\|_2^2, \quad (1)$$

where $\| \cdot \|_2$ is the Euclidean norm.

Objective function minimization is our objective.

Which objective function is optimal differs depending on the field, but once the objective function is determined, what needs to be done is aggregated into solving the objective function's minimization problem:

$$\underset{\theta \in \mathbb{R}^{d_{\text{param}}}}{\text{Minimize}} \mathcal{L}(\theta). \quad (2)$$

In this lecture, we learn a practical way to solve this minimization problem using the concept of a gradient vector, and explain why using a gradient vector is promising.

Brute Force Search is Impossible in Practice

In modern generative AI models with a huge number of parameters, brute force search is computationally impossible.

Definition (Brute Force Search)

We decide on a discretization width $\delta > 0$ and assume that each component of θ is restricted to a finite set of candidates $\{k\delta \mid k \in \mathbb{Z}\}$ (in practice, truncated to a finite range).

Brute force search is the method of performing a full search of $\mathcal{L}$ over the Cartesian product set of all candidate points and selecting the θ that gives the minimum value.

Impossible at all

Example

Suppose there are $d_{\text{param}} = 10^9$ parameters, and each component takes $2^8 = 256$ possible values (like float8).

The total number of candidate points is

$$256^{10^9} = 2^{8 \cdot 10^9} \approx 10^{(8 \cdot 10^9) \log_{10} 2} \approx 10^{2.4 \times 10^9},$$

an astronomical scale.

Even if one evaluation could be processed at 10^{15} FLOP and executed in parallel at 10^{18} FLOP per second (exa-scale), this is impossibly out of scale.

Therefore, global exhaustive search is not realistic.

From Global Search to Local Search

Since global search is impossible, we consider repeating a **local search**.

Local search:

- Operation of searching for a better parameter (where $\mathcal{L}$ is smaller) in the vicinity of the current parameter vector.
- Uses only information from the current parameter vector or its vicinity.

We start with an initial parameter vector $\theta^{[0]}$ (random or reasonably good) and repeat local search, updating

$$\theta^{[0]} \rightarrow \theta^{[1]} \rightarrow \theta^{[2]} \rightarrow \dots$$

to get closer to a better vector.

Local search

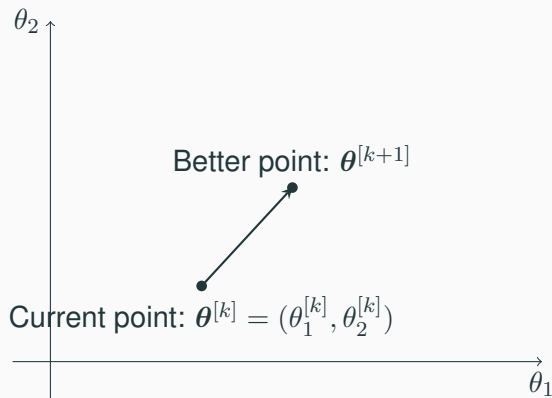


Figure 1: Conceptual diagram of local search from the current point $\theta^{[k]}$ in the (θ_1, θ_2) -plane.

Directional Derivatives

As an efficient local search, we adopt the idea of slightly updating the parameter vector in the direction that reduces $\mathcal{L}$ as efficiently as possible.

The rate of change of $\mathcal{L}$ when moving θ in the direction of a unit vector u is expressed by the **one-sided directional derivative**.

Definition (One-sided Directional Derivative)

Let $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$, $\theta \in \mathbb{R}^d$, and $u \in \mathbb{R}^d$ be a unit vector ($\|u\|_2 = 1$).

The **one-sided directional derivative** $D_u^+ \mathcal{L}(\theta)$ is

$$D_u^+ \mathcal{L}(\theta) := \lim_{t \rightarrow 0+} \frac{\mathcal{L}(\theta + tu) - \mathcal{L}(\theta)}{t} \quad (3)$$

(if the limit exists).

Remarks: directional derivatives

Remark

The one-sided directional derivative can be defined even for non-unit u , but then the “speed of advance” differs by direction.

If we want to compare directions fairly, we should restrict to unit vectors.

Remark

If $\mathcal{L}$ is differentiable at θ (definition later), then for any unit vector u , the limit as $t \rightarrow 0-$ also exists and coincides with the limit as $t \rightarrow 0+$.

In this case, we simply call it the **directional derivative** and may write $D_u \mathcal{L}(\theta)$.

Directional Derivatives and Their Graphs

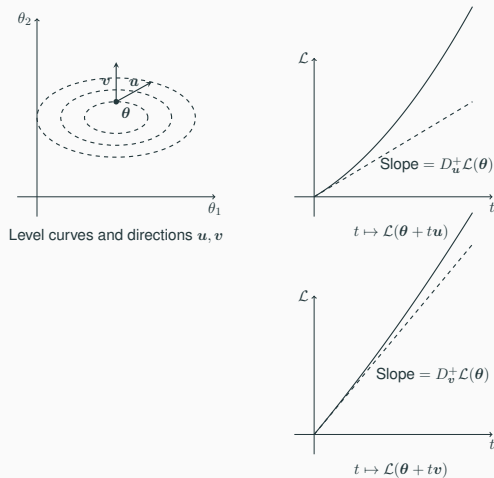


Figure 2: Graphs of $t \mapsto \mathcal{L}(\theta + tu)$ and $t \mapsto \mathcal{L}(\theta + tv)$ and their tangents at $t = 0$.

Optimizing the Directional Derivative

We want the direction that reduces $\mathcal{L}$ as efficiently as possible near the current parameter vector.

Since the one-sided directional derivative expresses exactly that efficiency, we want the direction that minimizes it.

Definition (Optimization of One-sided Directional Derivative)

We consider:

$$\text{Maximize:} \quad \underset{\|u\|_2=1}{\text{Maximize}} \quad D_u^+ \mathcal{L}(\theta), \quad (4)$$

$$\text{Minimize:} \quad \underset{\|u\|_2=1}{\text{Minimize}} \quad D_u^+ \mathcal{L}(\theta). \quad (5)$$

Under appropriate assumptions, the solution to this problem is given by the famous **gradient vector**.

Gradient vector

Definition (Gradient Vector)

When $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ has all partial derivatives at $\boldsymbol{\theta}$,

$$\frac{\partial \mathcal{L}}{\partial \theta_i}(\boldsymbol{\theta}) := \lim_{t \rightarrow 0} \frac{\mathcal{L}(\boldsymbol{\theta} + t\mathbf{e}_i) - \mathcal{L}(\boldsymbol{\theta})}{t},$$

the **gradient** is

$$\nabla \mathcal{L}(\boldsymbol{\theta}) := \begin{bmatrix} \partial \mathcal{L} / \partial \theta_1(\boldsymbol{\theta}) \\ \vdots \\ \partial \mathcal{L} / \partial \theta_d(\boldsymbol{\theta}) \end{bmatrix}.$$

Differentiability of Multivariate Functions

To relate gradients and directional derivatives, we use differentiability in the multivariate sense.

Definition (Differentiability of Multivariate Functions)

Let $\mathcal{L} : U \rightarrow \mathbb{R}$ on an open set $U \subset \mathbb{R}^d$ and $\theta \in U$.

If there exists a linear map $A : \mathbb{R}^d \rightarrow \mathbb{R}$ such that

$$\lim_{h \rightarrow 0} \frac{\mathcal{L}(\theta + h) - \mathcal{L}(\theta) - A(h)}{\|h\|_2} = 0, \quad (6)$$

we say that $\mathcal{L}$ is **differentiable** at θ , and call A the **derivative** $D\mathcal{L}(\theta)$.

Confirming the differentiability is practically tractable.

Many functions used in neural networks are differentiable (linear maps, common activations, etc.), and compositions of differentiable functions are differentiable:

Proposition (Differentiability of Composite Functions)

Let $U \subset \mathbb{R}^d$, $V \subset \mathbb{R}^m$ open, and $\mathcal{L}_1 : U \rightarrow V$, $\mathcal{L}_2 : V \rightarrow \mathbb{R}$.

If $\mathcal{L}_1$ is differentiable at $\theta \in U$ and $\mathcal{L}_2$ is differentiable at $\mathcal{L}_1(\theta) \in V$, then $\mathcal{L}_2 \circ \mathcal{L}_1$ is differentiable at θ and

$$D(\mathcal{L}_2 \circ \mathcal{L}_1)(\theta) = D\mathcal{L}_2(\mathcal{L}_1(\theta)) \circ D\mathcal{L}_1(\theta). \quad (7)$$

Steepest Ascent/Descent and the Gradient Vector

Theorem (Directions of Steepest Ascent/Descent)

Let $\mathcal{L} : U \rightarrow \mathbb{R}$ be differentiable on an open set $U \subset \mathbb{R}^d$, and $\theta \in U$. Assume all partial derivatives $\partial \mathcal{L} / \partial \theta_i(\theta)$ exist.

1. For any unit vector u , the one-sided directional derivative exists and

$$D_u^+ \mathcal{L}(\theta) = \nabla \mathcal{L}(\theta)^\top u.$$

2. If $\nabla \mathcal{L}(\theta) \neq 0$, the maximizer of (4) is $u^\star = \frac{\nabla \mathcal{L}(\theta)}{\|\nabla \mathcal{L}(\theta)\|_2}$, and the maximum value is $\|\nabla \mathcal{L}(\theta)\|_2$. If $\nabla \mathcal{L}(\theta) = 0$, then $D_u^+ \mathcal{L}(\theta) = 0$ for all unit u .
3. If $\nabla \mathcal{L}(\theta) \neq 0$, the minimizer of (5) is $u_\star = -\frac{\nabla \mathcal{L}(\theta)}{\|\nabla \mathcal{L}(\theta)\|_2}$, and the minimum value is $-\|\nabla \mathcal{L}(\theta)\|_2$. If $\nabla \mathcal{L}(\theta) = 0$, again $D_u^+ \mathcal{L}(\theta) = 0$ for all unit u .

Idea of the Proof (Steepest Ascent/Descent)

Sketch of proof idea:

- Differentiability implies

$$\mathcal{L}(\boldsymbol{\theta} + \boldsymbol{h}) = \mathcal{L}(\boldsymbol{\theta}) + A(\boldsymbol{h}) + r(\boldsymbol{h}),$$

where A is linear and $r(\boldsymbol{h})$ is small compared to $\|\boldsymbol{h}\|_2$.

- For a unit vector $\boldsymbol{u}$ and small t ,

$$\frac{\mathcal{L}(\boldsymbol{\theta} + t\boldsymbol{u}) - \mathcal{L}(\boldsymbol{\theta})}{t} = A(\boldsymbol{u}) + \frac{r(t\boldsymbol{u})}{t} \rightarrow A(\boldsymbol{u}).$$

- Thus, $D_{\boldsymbol{u}}^+ \mathcal{L}(\boldsymbol{\theta}) = A(\boldsymbol{u})$.
- By relating A to partial derivatives, we show $A(\boldsymbol{u}) = \nabla \mathcal{L}(\boldsymbol{\theta})^\top \boldsymbol{u}$.
- Maximizing/minimizing $\nabla \mathcal{L}^\top \boldsymbol{u}$ subject to $\|\boldsymbol{u}\|_2 = 1$ is done via the Cauchy–Schwarz inequality, yielding the claimed optimal directions.

Gradients are Orthogonal to Level Sets

One helpful geometric property: under suitable assumptions, the gradient is orthogonal to level sets of $\mathcal{L}$.

Proposition (Gradient is Orthogonal to Level Sets)

Assume $\mathcal{L} : U \rightarrow \mathbb{R}$ is differentiable on an open set $U \subset \mathbb{R}^d$.

*Fix $c \in \mathbb{R}$ and define the **level set***

$$S := \{\boldsymbol{\theta} \in U \mid \mathcal{L}(\boldsymbol{\theta}) = c\}.$$

Take $\boldsymbol{\theta}_0 \in S$ and assume $\nabla \mathcal{L}(\boldsymbol{\theta}_0) \neq \mathbf{0}$.

*If $\boldsymbol{w}$ is a **tangent vector** to S at $\boldsymbol{\theta}_0$ (i.e., there exists a C^1 curve γ in S with $\gamma(0) = \boldsymbol{\theta}_0$, $\gamma'(0) = \boldsymbol{w}$), then*

$$\nabla \mathcal{L}(\boldsymbol{\theta}_0)^\top \boldsymbol{w} = 0.$$

That is, the gradient vector is orthogonal to any tangent vector of the level set.

The gradient vector is orthogonal to the level set

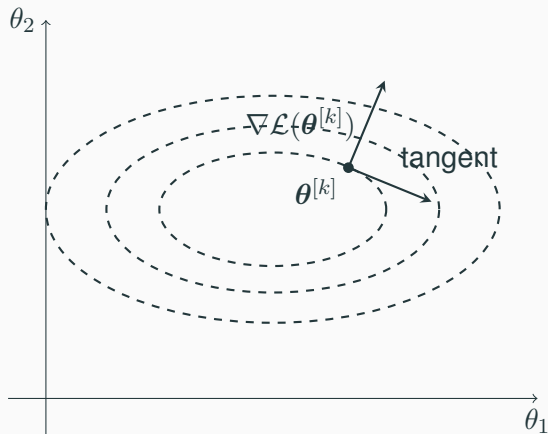


Figure 3: Gradient is perpendicular to the level curve of $\mathcal{L}$.

Steepest Descent Method

Since the negative gradient represents the direction of steepest local decrease, a natural idea is to repeatedly move in that direction.

Definition (Steepest Descent Method)

Given initial point $\theta^{[0]} \in \mathbb{R}^d$, number of steps $K \in \mathbb{Z}_{>0}$, and learning rates $(\eta^{[k]})_{k=0}^{K-1} \subset \mathbb{R}_{>0}$, we update

$$\theta^{[k+1]} := \theta^{[k]} - \eta^{[k]} \nabla \mathcal{L}(\theta^{[k]}), \quad k = 0, 1, \dots, K - 1. \quad (8)$$

This algorithm is called the **steepest descent method**.

In actual generative AI learning, we use large-scale data (big data) to define $\mathcal{L}$, which makes each exact gradient computation expensive.

Thus, the naive steepest descent method can be too costly, and we need more practical variants, which we discuss next.

Objective Functions with Big Data

Consider the case where the objective function depends on big data.

Suppose we use $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})_{i=1}^m$ and m is large, and the objective has the form

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \ell_{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})}(\boldsymbol{\theta}). \quad (9)$$

Then the gradient is

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \nabla \ell_{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})}(\boldsymbol{\theta}), \quad (10)$$

and the summation over i becomes a bottleneck.

Therefore, in large-scale learning, we usually employ algorithms that **do not** sum over all data at each parameter update.

We now introduce the general framework of stochastic gradient methods and two important instances: SGD and AdamW.

General Stochastic Gradient Methods

Definition (General Stochastic Gradient Methods)

At each step k , suppose we can obtain a random vector $\mathbf{g}^{[k]}$ satisfying

$$\mathbb{E}[\mathbf{g}^{[k]} \mid \mathcal{F}^{[k]}] = \nabla \mathcal{L}(\boldsymbol{\theta}^{[k]}),$$

where $\mathcal{F}^{[k]}$ represents past information.

A general update rule using all past $\mathbf{g}^{[0]}, \dots, \mathbf{g}^{[k]}$ is:

$$\mathbf{h}^{[k]} := \Phi_k(\mathbf{g}^{[0]}, \dots, \mathbf{g}^{[k]}), \quad \boldsymbol{\theta}^{[k+1]} := \Psi_k(\boldsymbol{\theta}^{[k]}, \mathbf{h}^{[k]}), \quad (11)$$

where Φ_k, Ψ_k are deterministic maps.

The unbiased estimator appears naturally in big data situations.

Example (Unbiased Minibatch Gradient)

In the situation of (9), take a uniform random minibatch $S^{[k]} \subset \{1, \dots, m\}$ of any size B , and define

$$\mathbf{g}^{[k]} := \frac{1}{|S^{[k]}|} \sum_{i \in S^{[k]}} \nabla \ell_{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})}(\boldsymbol{\theta}^{[k]}). \quad (12)$$

Then $\mathbb{E}[\mathbf{g}^{[k]} \mid \boldsymbol{\theta}^{[k]}] = \nabla \mathcal{L}(\boldsymbol{\theta}^{[k]})$ holds (assuming uniform sampling).

Stochastic Gradient Descent (SGD)

Definition (SGD)

Given learning rates $(\eta^{[k]})_{k \geq 0}$ and an unbiased estimate $\mathbf{g}^{[k]}$ at each step, we update:

$$\boldsymbol{\theta}^{[k+1]} := \boldsymbol{\theta}^{[k]} - \eta^{[k]} \mathbf{g}^{[k]}. \quad (13)$$

It is related to the classic Robbins–Monro stochastic approximation [4].

Gradient descent - answer

Question: How do we select a good parameter vector θ^* ?

Answer:

Gradient descent - answer

Question: How do we select a good parameter vector θ^* ?

Answer: When we use a parametric function which is differential almost everywhere, using a gradient descent is a good choice. If the problem is large-scale, we might want to use stochastic gradient descent method.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations Revisited

3. Key in efficient parameter selection on a large NN? - Gradient descent.

4. AdamW, Muon, ... Why do we need to learn so many?

5. Can we compute a gradient on a complicated NN? - General backpropagation

6. Summary

Outline

1. Introduction
2. Preliminaries: Mathematical Notations Revisited
3. Key in efficient parameter selection on a large NN? - Gradient descent.
- 4. AdamW, Muon, ... Why do we need to learn so many?**
5. Can we compute a gradient on a complicated NN? - General backpropagation
6. Summary

Question: In practical generative AI learning, advanced optimization algorithms are used, e.g., AdamW, Muon, etc. Why

Question: In practical generative AI learning, advanced optimization algorithms are used, e.g., AdamW, Muon, etc. Why do we need to learn them?

Why do we need to remember so many methods?

It would be ideal if there existed one excellent optimization algorithm working well for every function.

Why do we need to remember so many methods?

It would be ideal if there existed one excellent optimization algorithm working well for every function.

However, we can mathematically prove that it is impossible. No optimization algorithm is uniformly best, unless we limit the class of functions.

Why do we need to remember so many methods?

It would be ideal if there existed one excellent optimization algorithm working well for every function.

However, we can mathematically prove that it is impossible. No optimization algorithm is uniformly best, unless we limit the class of functions.

In real AI applications, we do not know the property of the real objective function. Hence,

Why do we need to remember so many methods?

It would be ideal if there existed one excellent optimization algorithm working well for every function.

However, we can mathematically prove that it is impossible. No optimization algorithm is uniformly best, unless we limit the class of functions.

In real AI applications, we do not know the property of the real objective function. Hence, trial and errors are necessary to find the best optimization algorithm. It is impossible to pick the algorithm only using theories.

AdamW: Adaptive Moment Estimation with Decoupled Weight Decay

Definition (AdamW [2])

Hyperparameters: $\beta_1, \beta_2 \in [0, 1)$, $\epsilon > 0$, learning rate $\eta^{[k]}$, decay coefficient $\lambda \geq 0$.

Let $\mathbf{g}^{[k]}$ be the minibatch gradient estimate. Initialize $\mathbf{m}^{[-1]} = \mathbf{0}$, $\mathbf{v}^{[-1]} = \mathbf{0}$.

For each $k = 0, 1, \dots$:

$$\mathbf{m}^{[k]} := \beta_1 \mathbf{m}^{[k-1]} + (1 - \beta_1) \mathbf{g}^{[k]}, \quad (14)$$

$$\mathbf{v}^{[k]} := \beta_2 \mathbf{v}^{[k-1]} + (1 - \beta_2) (\mathbf{g}^{[k]} \odot \mathbf{g}^{[k]}), \quad (15)$$

$$\widehat{\mathbf{m}}^{[k]} := \mathbf{m}^{[k]} / (1 - \beta_1^{k+1}), \quad (16)$$

$$\widehat{\mathbf{v}}^{[k]} := \mathbf{v}^{[k]} / (1 - \beta_2^{k+1}), \quad (17)$$

$$\boldsymbol{\theta}^{[k+1]} := \boldsymbol{\theta}^{[k]} - \eta^{[k]} \frac{\widehat{\mathbf{m}}^{[k]}}{\sqrt{\widehat{\mathbf{v}}^{[k]} + \epsilon}} - \eta^{[k]} \lambda \boldsymbol{\theta}^{[k]}. \quad (18)$$

Here $\odot$ is element-wise product, and division between vectors is element-wise.

Intuition behind AdamW update

Remark

- (14): first moment (moving average of gradients).
- (15): second moment (moving average of squared gradients).
- (16)–(17): initial bias correction.
- (18): per-coordinate adaptive learning rate via $\hat{v}^{[k]}$, plus **decoupled weight decay** $-\eta^{[k]}\lambda\theta^{[k]}$.

Muon: matrix-form parameter update rule with modified singular values.

Recent LLM training (e.g., Qwen 3.8, DeepSeek v4) uses **Muon**.

It applies $\mathcal{N}_5$, which makes the update matrix's singular values almost uniform.

Muon: matrix-form parameter update rule with modified singular values.

Recent LLM training (e.g., Qwen 3.8, DeepSeek v4) uses **Muon**.

It applies $\mathcal{N}_5$, which makes the update matrix's singular values almost uniform.

Definition (Muon (MomentUm Orthogonalized by Newton–Schulz))

Let the parameter takes a matrix form: $\mathbf{W}_t \in \mathbb{R}^{m \times n}$, $m \leq n$. Fix the hyperparameters $\beta (= 0.95)$, η , λ . Initialize the momentum buffer by $\mathbf{M}_0 = \mathbf{O}$

For each optimization step, get a stochastic gradient $\mathbf{G}_t \in \mathbb{R}^{m \times n}$ such that $\mathbf{G}_t = \mathbb{E} \nabla_{\mathbf{W}_t} \mathcal{L}_t$, and apply the following:

$$\mathbf{M}_{t+1} = \beta \mathbf{M}_t + (1 - \beta) \mathbf{G}_t, \quad (19)$$

$$\mathbf{H}_t = (1 - \beta) \mathbf{G}_t + \beta \mathbf{M}_{t+1}, \quad (20)$$

$$\mathbf{W}_{t+1} = (1 - \eta\lambda) \mathbf{W}_t - \eta \mathcal{N}_5(\mathbf{H}_t). \quad (21)$$

Muon: The Map $\mathcal{N}_5$ balances singular values.

Given $\mathbf{H} \in \mathbb{R}^{m \times n}$, $m \leq n$, initialize $\mathbf{X}_0 = \frac{\mathbf{H}}{\|\mathbf{H}\|_F + \varepsilon}$ for small $\varepsilon (= 10^{-7})$.

For $k = 0, \dots, 4$, compute

$$\mathbf{X}_{k+1} = a\mathbf{X}_k + b\mathbf{A}_k\mathbf{X}_k + c\mathbf{A}_k^2\mathbf{X}_k, \text{ where } \mathbf{A}_k = \mathbf{X}_k\mathbf{X}_k^\top. \quad (22)$$

Then, output $\mathcal{N}_5(\mathbf{H}) := \mathbf{X}_5$.

The map $\mathcal{N}_5$

Muon: The Map $\mathcal{N}_5$ balances singular values.

Given $\mathbf{H} \in \mathbb{R}^{m \times n}$, $m \leq n$, initialize $\mathbf{X}_0 = \frac{\mathbf{H}}{\|\mathbf{H}\|_F + \varepsilon}$ for small $\varepsilon (= 10^{-7})$.

For $k = 0, \dots, 4$, compute

$$\mathbf{X}_{k+1} = a\mathbf{X}_k + b\mathbf{A}_k\mathbf{X}_k + c\mathbf{A}_k^2\mathbf{X}_k, \text{ where } \mathbf{A}_k = \mathbf{X}_k\mathbf{X}_k^\top. \quad (22)$$

Then, output $\mathcal{N}_5(\mathbf{H}) := \mathbf{X}_5$.

The map $\mathcal{N}_5$ modifies the singular values without changing the singular vectors, with the odd polynomial $f(x) = ax + bx^3 + cx^5$ (default: $a = 3.4445, b = -4.7750, c = 2.0315$)

Specifically, if the SVD of $\mathbf{X}_0$ is $\mathbf{X}_0 = \mathbf{U} \text{diag}(\sigma_{1,0}, \sigma_{2,0}, \dots, \sigma_{m,0}) \mathbf{V}^\top$, then $\mathbf{X}_k = \mathbf{U} \text{diag}(\sigma_{1,k}, \sigma_{2,k}, \dots, \sigma_{m,k}) \mathbf{V}^\top$, where $\sigma_{i,k+1} = f(\sigma_{i,k})$ and thus $\sigma_{i,5} = (f \circ f \circ f \circ f \circ f)(\sigma_{i,0})$

Why not applying SVD?

We can directly modify the singular values if you apply SVD.

Why not applying SVD?

We can directly modify the singular values if you apply SVD.

However, SVD using GPU is not straightforward.

Why not applying SVD?

We can directly modify the singular values if you apply SVD.

However, SVD using GPU is not straightforward.

The advantage of $\mathcal{N}_5$ (Newton-Schulz iterations) is that we can modify the singular values only by dense matrix multiplication, which GPUs are good at.

Why not applying SVD?

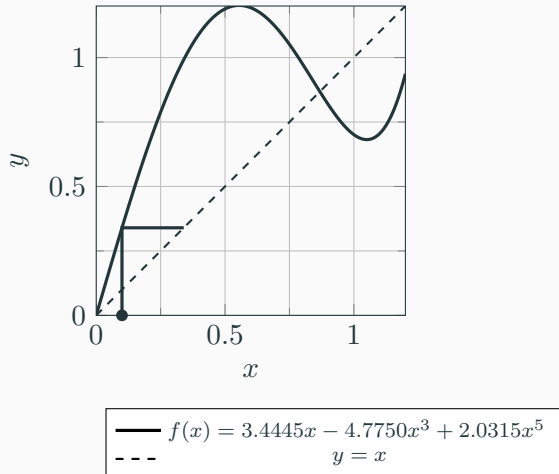
We can directly modify the singular values if you apply SVD.

However, SVD using GPU is not straightforward.

The advantage of $\mathcal{N}_5$ (Newton-Schulz iterations) is that we can modify the singular values only by dense matrix multiplication, which GPUs are good at.

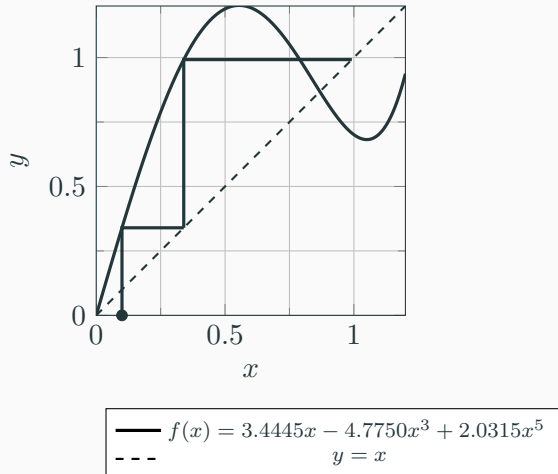
Not that the theoretical time computational cost of $\mathcal{N}_5$ is better.

Muon: Five Newton–Schulz Iterations



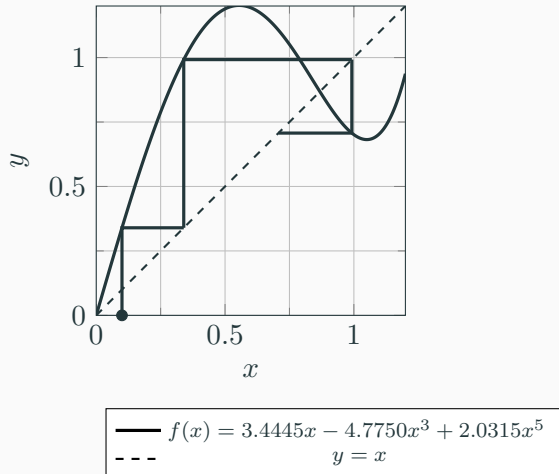
$$\sigma_{i,0} = 0.1 \quad \longmapsto \quad \sigma_{i,1} \approx 0.339695$$

Muon: Five Newton–Schulz Iterations



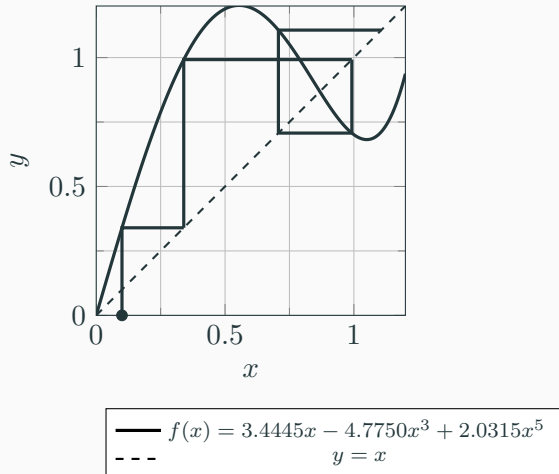
$$\sigma_{i,1} \approx 0.339695 \quad \longmapsto \quad \sigma_{i,2} \approx 0.992097$$

Muon: Five Newton–Schulz Iterations



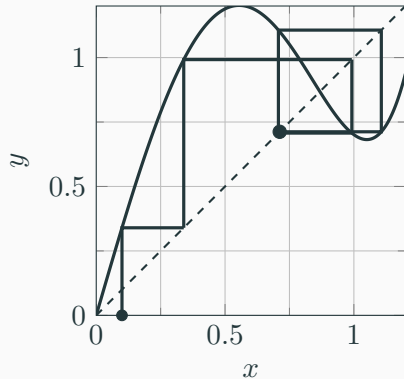
$$\sigma_{i,2} \approx 0.992097 \quad \longmapsto \quad \sigma_{i,3} \approx 0.707080$$

Muon: Five Newton–Schulz Iterations



$$\sigma_{i,3} \approx 0.707080 \quad \mapsto \quad \sigma_{i,4} \approx 1.106565$$

Muon: Five Newton–Schulz Iterations



— $f(x) = 3.4445x - 4.7750x^3 + 2.0315x^5$
- - - $y = x$

$$\sigma_{i,5} \approx 0.712120$$

Reality in optimization - answer

Question: In practical generative AI learning, advanced optimization algorithms are used, e.g., AdamW, Muon, etc. Why do we need to learn them?

Answer: This is because no optimization algorithm is always good. Practically,

Reality in optimization - answer

Question: In practical generative AI learning, advanced optimization algorithms are used, e.g., AdamW, Muon, etc. Why do we need to learn them?

Answer: This is because no optimization algorithm is always good. Practically, stochastic gradient descent variants using balanced update size for all the directions are popular.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations Revisited
3. Key in efficient parameter selection on a large NN? - Gradient descent.
- 4. AdamW, Muon, ... Why do we need to learn so many?**
5. Can we compute a gradient on a complicated NN? - General backpropagation
6. Summary

Outline

1. Introduction
2. Preliminaries: Mathematical Notations Revisited
3. Key in efficient parameter selection on a large NN? - Gradient descent.
4. AdamW, Muon, ... Why do we need to learn so many?
- 5. Can we compute a gradient on a complicated NN? - General backpropagation**
6. Summary

Question: Neural networks in modern AI applications are so complicated. Can we

Question: Neural networks in modern AI applications are so complicated. Can we still compute the gradient of the objective function on such a complicated NN?

Goal: Gradients for General Neural Networks

Up to now:

- If we can compute the gradient of the objective function for a given data point or minibatch,
- we can apply SGD or AdamW to minimize the objective.

Key question: Can we compute the gradient vector for various structured neural networks and objective functions used in generative AI?

Answer: Yes. For feedforward neural networks and objective functions depending on their node outputs, gradients can be computed efficiently via **backpropagation** [7], a dynamic programming algorithm.

Remark

Backpropagation is often introduced for MLPs (multi-layer perceptrons), but MLPs are rarely used directly in generative AI.

MLP Structure

Definition (MLP Structure)

An MLP with input dimension d_0 and number of layers $L \in \mathbb{Z}_{>0}$ is defined as follows:

For each layer $\ell \in [1, L]_{\mathbb{Z}}$:

- Weight matrix $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_\ell, d_{\ell-1}}$,
- Bias vector $\mathbf{b}^{(\ell)} \in \mathbb{R}^{d_\ell}$,
- Differentiable activation function $\varphi^{(\ell)} : \mathbb{R}^{d_\ell} \rightarrow \mathbb{R}^{d_\ell}$.

MLP Forward propagation

Definition (MLP Forward Propagation)

Given an input $\mathbf{x} \in \mathbb{R}^{d_0}$, define

$$\mathbf{h}^{(0)} := \mathbf{x}.$$

For $\ell \in [1, L]_{\mathbb{Z}}$:

$$\mathbf{z}^{(\ell)} := \mathbf{W}^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)} \in \mathbb{R}^{d_\ell}, \quad (23)$$

$$\mathbf{h}^{(\ell)} := \varphi^{(\ell)}(\mathbf{z}^{(\ell)}) \in \mathbb{R}^{d_\ell}. \quad (24)$$

The final output

$$\mathbf{h}^{(L)} = f_{(\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(1)}, \dots, \mathbf{b}^{(L)})}(\mathbf{x})$$

is the MLP output.

MLP forward propagation intuition

Remark

Each layer performs an affine transformation (linear map plus bias) followed by a non-linear activation. Repeating this constructs a complex non-linear map $f_{(\cdot)}$ from x to $h^{(L)}$.

Backpropagation for MLPs: Setup

Fix the input x and consider a scalar objective function $\mathcal{J} = \mathcal{J}(\mathbf{h}^{(L)}) \in \mathbb{R}$ depending on all parameters $\Theta := (\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(1)}, \dots, \mathbf{b}^{(L)})$.

Definition (MLP Backpropagation)

The backpropagation algorithm computes $\partial \mathcal{J} / \partial \mathbf{W}^{(\ell)}$ and $\partial \mathcal{J} / \partial \mathbf{b}^{(\ell)}$:

- **Forward pass:** compute and store $\mathbf{z}^{(\ell)}, \mathbf{h}^{(\ell)}$ using (23)–(24).
- **Initialization** (output layer): $\delta^{(L)} := \frac{\partial \mathcal{J}}{\partial \mathbf{h}^{(L)}} \odot \varphi^{(L)'}(\mathbf{z}^{(L)})$.
- **Backward recursion:** for $\ell = L - 1, \dots, 1$,

$$\delta^{(\ell)} := (\mathbf{W}^{(\ell+1)\top} \delta^{(\ell+1)}) \odot \varphi^{(\ell)'}(\mathbf{z}^{(\ell)}), \quad (25)$$

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(\ell)}} := \delta^{(\ell)} \mathbf{h}^{(\ell-1)\top}, \quad (26)$$

$$\frac{\partial \mathcal{J}}{\partial \mathbf{b}^{(\ell)}} := \delta^{(\ell)}. \quad (27)^{56/90}$$

MLP backpropagation intuition

Remark

$\delta^{(\ell)}$ corresponds to $\partial \mathcal{J} / \partial z^{(\ell)}$. Equation for $\delta^{(\ell)}$ implements the chain rule, passing gradients from layer $\ell + 1$ back to ℓ , then multiplying by activation derivatives. The gradients w.r.t. $\mathbf{W}^{(\ell)}$ and $\mathbf{b}^{(\ell)}$ come from differentiating the affine relation $z^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}$.

Correctness of MLP Backpropagation

Theorem (Correctness of MLP Backprop)

Assume each activation $\varphi^{(\ell)}$ is differentiable at $z^{(\ell)}$, and $\mathcal{J}$ is differentiable at $\mathbf{h}^{(L)}$.

Define $F(\Theta) := \mathcal{J}(\mathbf{h}^{(L)}(\Theta))$.

Then the matrices/vectors from Definition 17 satisfy, for all ℓ and indices i, j :

$$\left(\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(\ell)}} \right)_{i,j} = \frac{\partial F}{\partial w_{i,j}^{(\ell)}}, \quad (28)$$

$$\left(\frac{\partial \mathcal{J}}{\partial \mathbf{b}^{(\ell)}} \right)_i = \frac{\partial F}{\partial b_i^{(\ell)}}. \quad (29)$$

That is, the algorithm yields the correct partial derivatives of F .

General Feedforward Neural Network as a DAG

We generalize from MLPs to arbitrary feedforward networks defined on directed acyclic multigraphs (DAGs).

Definition (General Feedforward Neural Network)

The **architecture** is a tuple:

- **Computation Graph:** DAG $G = (\mathcal{V}, \mathcal{E}, \text{Tail}, \text{Head})$ with nodes $\mathcal{V} \subset \mathbb{Z}_{>0}$ topologically ordered and $\text{Tail}(e) < \text{Head}(e)$ for all e .
- **Input/Output dimensions:** $d_{\text{input}}, d_{\text{output}}$.
- **Node specification:** Each $v \in \mathcal{V}$ has
 - activation $g^{(v)} : \mathbb{R}^{d_{\text{arg}}^{(v)}} \rightarrow \mathbb{R}^{d_{\text{ret}}^{(v)}}$,
 - argument connection tuple $\text{NodeSrc}^{(v)}$ specifying sources from input or incoming edges.

General Feedforward Neural Network as a DAG

Definition (General Feedforward Neural Network)

- **Edge specification:** Each $e \in \mathcal{E}$ has $\text{EdgeSrc}(e)$ specifying which component of the tail node output it takes.
- **Parameter specification:** Parameter dimension d_{param} and surjection $\text{Weight} : \mathcal{E} \rightarrow [1, d_{\text{param}}]\mathbb{Z}$, allowing weight sharing.
- **Output specification:** OutSrc maps output indices to node-output components.

Given $\theta \in \mathbb{R}^{d_{\text{param}}}$, $f_{\theta} : \mathbb{R}^{d_{\text{input}}} \rightarrow \mathbb{R}^{d_{\text{output}}}$ is defined by evaluating each node v :

$$a_m^{(v)} = \begin{cases} x_j & \text{if } \text{NodeSrc}_m^{(v)} = j, \\ \theta_{\text{Weight}(e)} r_{\text{EdgeSrc}(e)}^{(\text{Tail}(e))} & \text{if } \text{NodeSrc}_m^{(v)} = e, \end{cases} \quad (30)$$

$$\mathbf{r}^{(v)} := \mathbf{g}^{(v)}(\mathbf{a}^{(v)}), \quad (\text{output}). \quad (31)$$

Forward Propagation on a General DAG

Definition (Forward Propagation in Topological Order)

List nodes as $\mathcal{V} = \{v_1, \dots, v_N\}$ in a topological order.

Compute for $i = 1, \dots, N$:

- inputs $\mathbf{a}^{(v_i)}$ by collecting from $\mathbf{x}$ or parent nodes,
- outputs $\mathbf{r}^{(v_i)} = \mathbf{g}^{(v_i)}(\mathbf{a}^{(v_i)})$.

Then construct $\mathbf{y} = \mathbf{f}_\theta(\mathbf{x})$ from the specified node-output components.

This procedure is called forward propagation on the general neural network.

Remark

Topological ordering ensures that all inputs to a node are available when it is evaluated. This generalizes layer-wise evaluation in MLPs to arbitrary graph structures.

Backpropagation on a General DAG: Setup

We extend backpropagation to this general DAG.

Consider a scalar objective function

$$\mathcal{J} = \mathcal{J}(\mathbf{y}) \in \mathbb{R}, \quad \mathbf{y} = f_{\theta}(\mathbf{x}).$$

Introduce an **objective node** $v_{\mathcal{J}}$ as the last node in the topological order, with activation $g^{(v_{\mathcal{J}})}$ applying $\mathcal{J}$ to $\mathbf{y}$.

For each node v , we will maintain

$$\delta^{(v)} := \frac{\partial \mathcal{J}}{\partial \mathbf{r}^{(v)}}.$$

We also maintain gradient accumulators G_p for each parameter index p .

Backpropagation on a General DAG: Algorithm

Definition (Backpropagation on General DAG)

- **Forward pass:** Compute and store $\mathbf{a}^{(v)}$, $\mathbf{r}^{(v)}$ for all v in topological order and $\mathcal{J}$ at $v_{\mathcal{J}}$.
- **Initialization:** For all v , set $\delta^{(v)} = 0$. For the objective node, set

$$\delta^{(v_{\mathcal{J}})} := 1.$$

For each parameter index p , set $G_p := 0$.

Backpropagation on a General DAG: Algorithm ii

Definition (Backpropagation on General DAG)

- **Backprop loop:** Process nodes in reverse topological order $v_N, \dots, v_1$; denote current node by w .
 - Let $\mathbf{J}^{(w)}(\mathbf{a}^{(w)}) := \frac{\partial \mathbf{g}^{(w)}}{\partial \mathbf{a}^{(w)}}$ be the Jacobian. Define $\gamma^{(w)} := \frac{\partial \mathcal{J}}{\partial \mathbf{a}^{(w)}} = \mathbf{J}^{(w)}(\mathbf{a}^{(w)})^\top \boldsymbol{\delta}^{(w)}$.
 - For each argument index m of node w :
 1. If $\text{NodeSrc}_m^{(w)}$ is an input component, do nothing.
 2. If $\text{NodeSrc}_m^{(w)} = e$ is an incoming edge from $u = \text{Tail}(e)$, with parameter index $p = \text{Weight}(e)$ and source component $\text{EdgeSrc}(e)$, then:

$$G_p \leftarrow G_p + \gamma_m^{(w)} r_{\text{EdgeSrc}(e)}^{(u)}, \quad (32)$$

$$\delta_{\text{EdgeSrc}(e)}^{(u)} \leftarrow \delta_{\text{EdgeSrc}(e)}^{(u)} + \gamma_m^{(w)} \theta_p. \quad (33)$$

- **Output:** After the loop, set $\partial \mathcal{J} / \partial \theta_p := G_p$.

Backpropagation intuition

Remark

$\delta^{(v)}$ is the gradient w.r.t. node output $\mathbf{r}^{(v)}$, and $\gamma^{(w)}$ is the gradient w.r.t. node input $\mathbf{a}^{(w)}$.

By propagating δ backward through Jacobians and accumulating contributions along edges, we implement the chain rule on the DAG. Weight sharing is handled by summing contributions into the same G_p .

Correctness of Backpropagation on General DAG

Theorem (Correctness on General DAG)

Assume each node activation $g^{(v)}$ is differentiable at $\alpha^{(v)}$, and the objective node activation at $\alpha^{(v_{\mathcal{J}})}$ is differentiable.

For fixed input x , define

$$F(\theta) := \mathcal{J}(f_{\theta}(x)).$$

Then F is differentiable w.r.t. θ , and for each p ,

$$G_p = \frac{\partial F}{\partial \theta_p}.$$

That is, the algorithm produces the correct gradient vector $\nabla_{\theta} F$.

Worked Example: Network Structure

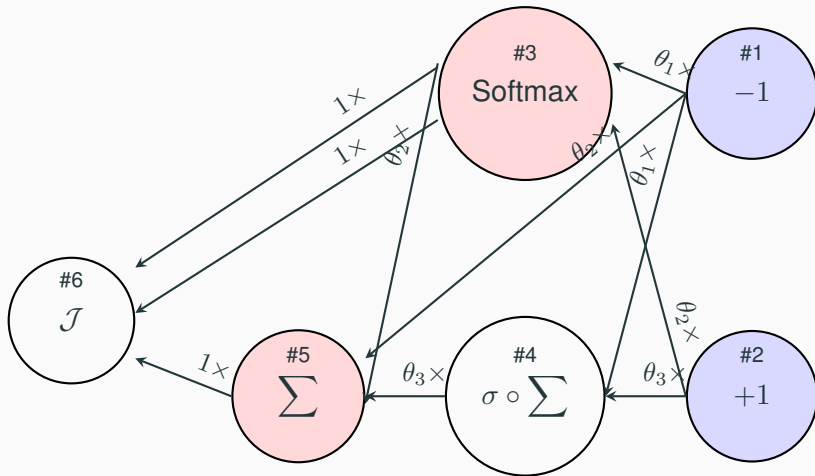


Figure 4: Example network structure (with objective node 6). Information flows from right to left.

Exercise on Example i

Consider a network with a node sequence $(1, 2, 3, 4, 5)$, and additionally an objective function node 6 (Figure ??). The edge and parameter correspondence is as follows.

$$(1, 3_1) : \theta_1, \quad (2, 3_2) : \theta_2, \quad (1, 4) : \theta_1, \quad (2, 4) : \theta_3, \quad (1, 5) : \theta_2, \quad (3_1, 5) : \theta_2, \quad (4, 5) : \theta_3. \quad (34)$$

The activation functions are defined as follows.

- Node 1: Input node (input value -1), Node 2: Input node (input value $+1$).
- Node 3: Softmax (2D input, 2D output).
- Node 4: Apply standard sigmoid $\sigma(z) = 1/(1 + e^{-z})$ after summing the inputs.

Exercise on Example ii

- Node 5: Linear node that sums the inputs (identity function).
- Node 6: Calculates the sum of the cross entropy $-\log s_2$ for data $(0, 1)$ with respect to the 2D output $s = (s_1, s_2)$ of node 3, and the square y_5^2 of the node 5 output y_5 .

Exercise on Example iii

Forward Propagation (Topological order $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$) is.

$$r^{(1)} = -1, \quad r^{(2)} = +1, \quad (35)$$

$$\text{Node 3: } z_1 = \theta_1 r^{(1)}, \quad z_2 = \theta_2 r^{(2)}, \quad s_j = \frac{e^{z_j}}{e^{z_1} + e^{z_2}}, \quad j = 1, 2, \quad (36)$$

$$\text{Node 4: } u = \theta_1 r^{(1)} + \theta_3 r^{(2)}, \quad y_4 = \sigma(u), \quad (37)$$

$$\text{Node 5: } y_5 = \theta_2 r^{(1)} + \theta_2 s_1 + \theta_3 y_4, \quad (38)$$

$$\text{Node 6: } \mathcal{J} = -\log s_2 + y_5^2. \quad (39)$$

Below, we apply the general backpropagation of Definition ?? to this specific example. We follow the calculations at each node according to the reverse topological sort order $6 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$.

Exercise on Example iv

Processing Node 6 (Objective Function Node).

Since the output of node 6 is $\mathcal{J}$ itself

$$\delta^{(6)} = \frac{\partial \mathcal{J}}{\partial \mathcal{J}} = 1 \quad (40)$$

. The input to node 6 is (s_1, s_2, y_5) , and

$$\frac{\partial \mathcal{J}}{\partial s_1} = 0, \quad \frac{\partial \mathcal{J}}{\partial s_2} = -\frac{1}{s_2}, \quad \frac{\partial \mathcal{J}}{\partial y_5} = 2y_5 \quad (41)$$

. This corresponds to $\gamma^{(6)}$ in Definition ?? and gives the initial gradient for the output (s_1, s_2) of node 3 and the output y_5 of node 5.

Exercise on Example v

Processing Node 5 (Σ).

The gradient with respect to the output of node 5 is, from Equation (41),

$$\delta^{(5)} = \frac{\partial \mathcal{J}}{\partial y_5} = 2y_5. \quad (42)$$

Node 5 is a linear node that takes the sum of the inputs $(\theta_2 r^{(1)}, \theta_2 s_1, \theta_3 y_4)$, so the gradient with respect to the inputs is

$$\frac{\partial \mathcal{J}}{\partial \theta_2 r^{(1)}} = 2y_5, \quad \frac{\partial \mathcal{J}}{\partial \theta_2 s_1} = 2y_5, \quad \frac{\partial \mathcal{J}}{\partial \theta_3 y_4} = 2y_5. \quad (43)$$

Exercise on Example vi

From here, according to Equation (??) and Equation (??) of Definition ??, we propagate the gradient to the parameters and parent nodes through edges (1, 5), (3₁, 5), and (4, 5). Specifically

$$\text{Edge (1, 5) : } G_2 \leftarrow G_2 + (2y_5) \cdot r^{(1)}, \quad \delta^{(1)} \leftarrow \delta^{(1)} + (2y_5)\theta_2, \quad (44)$$

$$\text{Edge (3}_1\text{, 5) : } G_2 \leftarrow G_2 + (2y_5) \cdot s_1, \quad \delta_1^{(3)} \leftarrow \delta_1^{(3)} + (2y_5)\theta_2, \quad (45)$$

$$\text{Edge (4, 5) : } G_3 \leftarrow G_3 + (2y_5) \cdot y_4, \quad \delta^{(4)} \leftarrow \delta^{(4)} + (2y_5)\theta_3. \quad (46)$$

Processing Node 4 ($\sigma \circ \Sigma$).

Exercise on Example vii

The gradient with respect to the output of node 4 is given by $\delta^{(4)}$ in Equation (46). The activation of node 4 is $y_4 = \sigma(u)$, $u = \theta_1 r^{(1)} + \theta_3 r^{(2)}$, so from the single-variable chain rule

$$\frac{\partial \mathcal{J}}{\partial u} = \frac{\partial \mathcal{J}}{\partial y_4} \sigma'(u) = \delta^{(4)} \cdot \sigma(u)(1 - \sigma(u)). \quad (47)$$

Furthermore, from the parameter dependency of u

$$\frac{\partial \mathcal{J}}{\partial \theta_1} \leftarrow \frac{\partial \mathcal{J}}{\partial \theta_1} + \frac{\partial \mathcal{J}}{\partial u} \cdot r^{(1)} = G_1 + \frac{\partial \mathcal{J}}{\partial u} r^{(1)}, \quad (48)$$

$$\frac{\partial \mathcal{J}}{\partial \theta_3} \leftarrow \frac{\partial \mathcal{J}}{\partial \theta_3} + \frac{\partial \mathcal{J}}{\partial u} \cdot r^{(2)} = G_3 + \frac{\partial \mathcal{J}}{\partial u} r^{(2)}, \quad (49)$$

Exercise on Example viii

. Also, the gradients to input nodes 1, 2 are updated as

$$\delta^{(1)} \leftarrow \delta^{(1)} + \frac{\partial \mathcal{J}}{\partial u} \theta_1, \quad (50)$$

$$\delta^{(2)} \leftarrow \delta^{(2)} + \frac{\partial \mathcal{J}}{\partial u} \theta_3. \quad (51)$$

Processing Node 3 (Softmax).

At node 3, softmax is applied to the input (z_1, z_2) to obtain the output (s_1, s_2) . From the standard derivation for the combination of cross entropy $-\log s_2$ and softmax (chain rule using the Jacobian as a matrix)

$$\frac{\partial \mathcal{J}}{\partial z_j} = s_j - t_j, \quad t = (0, 1) \quad (52)$$

Exercise on Example ix

holds. Here, since there is an additional gradient of $2y_5$ in the s_1 direction as a contribution from node 5 (Equation (45)), applying Equation (??) of Definition ?? gives

$$\frac{\partial \mathcal{J}}{\partial z_1} = s_1 + 2y_5 \cdot s_1(1 - s_1), \quad (53)$$

$$\frac{\partial \mathcal{J}}{\partial z_2} = (s_2 - 1) - 2y_5 \cdot s_1 s_2. \quad (54)$$

This corresponds to $\delta_1^{(3)}, \delta_2^{(3)}$.

Exercise on Example x

The gradients to the parameters and input nodes are, from a form similar to Equation (??), updated as

$$\frac{\partial \mathcal{J}}{\partial \theta_1} \leftarrow \frac{\partial \mathcal{J}}{\partial \theta_1} + \frac{\partial \mathcal{J}}{\partial z_1} \cdot r^{(1)}, \quad (55)$$

$$\frac{\partial \mathcal{J}}{\partial \theta_2} \leftarrow \frac{\partial \mathcal{J}}{\partial \theta_2} + \frac{\partial \mathcal{J}}{\partial z_2} \cdot r^{(2)}, \quad (56)$$

$$\delta^{(1)} \leftarrow \delta^{(1)} + \frac{\partial \mathcal{J}}{\partial z_1} \theta_1, \quad (57)$$

$$\delta^{(2)} \leftarrow \delta^{(2)} + \frac{\partial \mathcal{J}}{\partial z_2} \theta_2. \quad (58)$$

Processing Node 2 and Node 1 (Input Nodes).

Exercise on Example xi

Node 2 and Node 1 are nodes that only receive external inputs and have no further parent nodes. Therefore, when they are reached, $\delta^{(1)}$, $\delta^{(2)}$ have become their final values, and these can be used to read the sensitivity to the inputs.

Finally, by aggregating all of Equation (48), (49), (55), (56), and the contributions from node 5 (44)–(46), the gradients with respect to the parameters $\theta_1, \theta_2, \theta_3$

$$\frac{\partial \mathcal{J}}{\partial \theta_1} = \frac{\partial \mathcal{J}}{\partial z_1} \cdot r^{(1)} + \frac{\partial \mathcal{J}}{\partial u} \cdot r^{(1)}, \quad (59)$$

$$\frac{\partial \mathcal{J}}{\partial \theta_2} = \frac{\partial \mathcal{J}}{\partial z_2} \cdot r^{(2)} + (2y_5)r^{(1)} + (2y_5)s_1, \quad (60)$$

$$\frac{\partial \mathcal{J}}{\partial \theta_3} = \frac{\partial \mathcal{J}}{\partial u} \cdot r^{(2)} + (2y_5)y_4 \quad (61)$$

Exercise on Example xii

are obtained. By substituting $r^{(1)} = -1$, $r^{(2)} = +1$ here, a completely specific gradient is calculated. This series of procedures is a concrete calculation showing that the general backpropagation algorithm of Definition ?? gives the correct gradient even in a specific example.

Note: What Happens with Non-differentiable Functions? i

Practical neural networks use many differentiable functions, but also non-differentiable functions like ReLU. In fact, even if not differentiable, the functions practically used as activation functions in neural networks are mostly locally Lipschitz, and for locally Lipschitz functions, the **Clarke subdifferential**, which is a generalization of the gradient vector, can be defined. This is a set of vectors. For example, the subdifferential $\partial^\circ \text{ReLU}$ of ReLU is

$$\partial^\circ \text{ReLU}(x) = \begin{cases} \{0\} & \text{if } x < 0, \\ [0, 1] & \text{if } x = 0, \\ \{1\} & \text{if } x > 0, \end{cases} \quad (62)$$

, which matches intuition.

Note: What Happens with Non-differentiable Functions? ii

The problem lies in the chain rule. Although we will not make a rigorous assertion here, the subdifferential of a composite function is a subset of the set consisting of elements obtained by applying the chain rule by arbitrarily taking elements of the subdifferential of each function. It is important that it is a subset, but not equal as a set. In other words, backpropagation based on the chain rule has validity for non-differentiable functions in the sense that if we appropriately choose an element of the subdifferential at each step, we can obtain one element of the overall subdifferential.

On the other hand, if we cannot appropriately choose an element of the subgradient at each step, there is no guarantee that the obtained vector will be in the overall subdifferential. It is difficult to appropriately choose an element of the

Note: What Happens with Non-differentiable Functions? iii

subgradient in the general case. That is where it is theoretically difficult. However, in reality, in PyTorch, the gradient of ReLU at the origin is defined by convention as 0, and moreover, the backpropagation method determined by it works well empirically.

General backpropagation - answer

Question: Neural networks in modern AI applications are so complicated. Can we still compute the gradient of the objective function on such a complicated NN?

Answer: YES! The backpropagation algorithm works even on a complicated neural network defined on a general directed acyclic graph!

Outline

1. Introduction
2. Preliminaries: Mathematical Notations Revisited
3. Key in efficient parameter selection on a large NN? - Gradient descent.
4. AdamW, Muon, ... Why do we need to learn so many?
- 5. Can we compute a gradient on a complicated NN? - General backpropagation**
6. Summary

Outline

1. Introduction
2. Preliminaries: Mathematical Notations Revisited
3. Key in efficient parameter selection on a large NN? - Gradient descent.
4. AdamW, Muon, ... Why do we need to learn so many?
5. Can we compute a gradient on a complicated NN? - General backpropagation

6. Summary

Summary

Summary

- The gradient descent realizes efficient local search avoiding a brute force search. Stochastic gradient variants works efficiently in large data situation. Backpropagation allows us to compute the stochastic gradient of a general loss function and a general NN.

Summary

- The gradient descent realizes efficient local search avoiding a brute force search. Stochastic gradient variants works efficiently in large data situation. Backpropagation allows us to compute the stochastic gradient of a general loss function and a general NN.
- Without any assumption on the objective function, no optimization algorithm is uniformly best. In practical AI application situations, often no assumptions on the objective function is known, so empirical studies are more or less necessary to select the best optimization algorithm.

Summary

- The gradient descent realizes efficient local search avoiding a brute force search. Stochastic gradient variants works efficiently in large data situation. Backpropagation allows us to compute the stochastic gradient of a general loss function and a general NN.
- Without any assumption on the objective function, no optimization algorithm is uniformly best. In practical AI application situations, often no assumptions on the objective function is known, so empirical studies are more or less necessary to select the best optimization algorithm.
- Both AdamW and Muon tries to modify the update direction. They are stochastic gradient descent variants using balanced update size for all the directions.

References i

- [1] Thomas M. Cover and Joy A. Thomas.
Elements of Information Theory.
Wiley, 2 edition, 2006.
- [2] Ilya Loshchilov and Frank Hutter.
Decoupled weight decay regularization.
In International Conference on Learning Representations (ICLR) 2019, 2019.

References ii

- [3] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever.

Learning transferable visual models from natural language supervision.

In Proceedings of the 38th International Conference on Machine Learning (ICML) 2021, 2021.

- [4] Herbert Robbins and Sutton Monro.

A stochastic approximation method.

The Annals of Mathematical Statistics, 22(3):400–407, 1951.

References iii

- [5] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer.

High-resolution image synthesis with latent diffusion models.

In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) 2022, 2022.

- [6] Olaf Ronneberger, Philipp Fischer, and Thomas Brox.

U-net: Convolutional networks for biomedical image segmentation.

In Medical Image Computing and Computer-Assisted Intervention (MICCAI) 2015, 2015.

References iv

- [7] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams.
Learning representations by back-propagating errors.
Nature, 323(6088):533–536, 1986.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin.
Attention is all you need.
In Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 2017.